



Software Requirements Specification (SRS)

Project: Personal Loan Management System (Bank — Monolithic Spring Boot)

Version: 1.0

Date: *[fill date]*

Prepared by: Business Analyst / System Architect

1. Introduction

1.1 Purpose of this document

This SRS describes the functional and non-functional requirements for the **Personal Loan Management System (PLMS)** — a monolithic Spring Boot application designed for a bank to manage end-to-end personal loan lifecycle (application → verification → underwriting → sanction → disbursement → repayment). This document is intended for product owners, business analysts, developers, testers, operations, and stakeholders.

1.2 Scope of the system

PLMS provides digital loan application and processing for Personal Loans only. It includes customer onboarding, document upload/verification, credit score integration, loan officer & underwriter workflows, sanction & disbursement, EMI generation and payments, collection workflows, and admin capabilities.

1.3 Definitions, acronyms, abbreviations

- PLMS — Personal Loan Management System
- API — Application Programming Interface
- EMI — Equated Monthly Installment
- KYC — Know Your Customer
- CIBIL — Credit Information Bureau (credit score)
- NEFT/IMPS — Bank transfer methods
- JWT — JSON Web Token
- SLA — Service Level Agreement

1.4 References

- BRS – Personal Loan Management System (Version 1.0)
 - RBI KYC and lending guidelines (applicable local regulations)
 - IEEE 830 / ISO/IEC/IEEE 29148 guidelines
-

2. Overall Description

2.1 Product perspective

Monolithic backend (Spring Boot) + single-page frontend (Angular/React).
Monolith contains modules (packages): authentication, customer, loan, document, verification, underwriting, disbursement, payments, collection, admin, reporting, notifications.

2.2 User classes and characteristics

- **Customer:** End user; basic computer/mobile skills.
- **Loan Officer:** Bank staff; reviews applications.
- **Verification Team:** Bank staff; verifies documents/KYC.
- **Underwriter:** Bank staff; risk evaluation and final approval.
- **Disbursement Officer:** Bank staff; processes payouts.
- **Collection Team:** Bank staff; manages overdue accounts.
- **Admin / Super Admin:** System configuration and product management.

All internal users access via secure VPN or bank intranet; customers access via bank portal.

2.3 Operating environment

- Backend: Java 17, Spring Boot 3.x
- Database: MySQL 8
- Frontend: Angular 14+ or React 18+
- Build: Maven
- VCS: Git (GitHub/GitLab)
- Hosting: AWS (EC2 + RDS)
- TLS termination via bank-managed load balancer

2.4 Design & implementation constraints

- Must comply with RBI KYC & data retention rules.
- Aadhaar/PAN must be masked in UI and logs (PII protection).
- Document storage in encrypted object store (on-prem S3-like or AWS S3 with SSE).
- Single monolithic WAR/JAR for deployment.
- Database schema must support ACID transactions for loan operations.

2.5 User documentation

- User manuals for each role (Customer, LO, Underwriter, Admin).
 - API docs (OpenAPI/Swagger).
 - Deployment and runbook for operations.
-

3. System Features (Functional Requirements)

Each feature contains: description, actors, preconditions, main flow, exceptions, postconditions.

3.1 Authentication & Authorization (FR-AUTH)

Description: Secure registration, login, password reset, role-based access control.

Actors: Customer, Bank users, Admin.

Preconditions: User device has internet.

Main flow:

1. Customer registers with mobile number & email.
2. System sends OTP (SMS) — verify.
3. User sets password; 2FA optional (OTP on login).
4. Users authenticate and receive JWT.

Exceptions: OTP expired; account locked after N failed attempts.

Notes: Passwords hashed with bcrypt; JWT expiry configurable.

3.2 Customer Onboarding (FR-CO)

Description: Collect basic profile and KYC details.

Actors: Customer.

Data: Name, DOB, address, PAN, Aadhaar (partial), employment, income.

Validation: PAN format, mobile format, DOB limits (age 21–58).

Postcondition: Customer profile created and pending KYC verification.

3.3 Loan Application (FR-LA)

Description: Multi-step loan application form with validations.

Actors: Customer.

Fields: Loan amount, tenure, purpose, employment details, monthly income, bank account details.

Business rules: Min ₹25,000; Max ₹10,00,000; Tenure 12–60 months.

Process: Applicant completes form → system stores application with status

SUBMITTED.

3.4 Document Upload & Management (FR-DOC)

Description: Upload required documents (PAN, Aadhaar, Salary Slips, Bank statements).

Actors: Customer, Verification Team.

Constraints: File types (PDF/JPG/PNG), max 5MB per file.

Storage: Documents stored encrypted, reference in DB (document_id, path, checksum, upload_date, uploader).

Versioning: Re-upload creates new revision record.

3.5 Credit Bureau Integration (FR-CIBIL)

Description: Fetch credit score and bureau report.

Actors: System (automated), CIBIL API.

Flow: On application submit, PLMS queries CIBIL with customer identifiers.

Business rule: Score < 700 → flag for manual review or auto-reject per config.

Error handling: If API fails, mark application **PENDING_SCORE** and retry policy applies.

3.6 KYC Verification (FR-KYC)

Description: Verification Team validates uploaded documents and KYC fields.

Actors: Verification Team.

Tasks: Auto checks (PAN/Aadhaar APIs), manual cross-check.

Statuses: `KYC_PENDING` , `KYC_VERIFIED` , `KYC_REJECTED` .

Audit: Every action logged with timestamp and user id.

3.7 Loan Officer Review (FR-LO)

Description: Loan officers view and take initial action (approve/reject/request more docs).

Actors: Loan Officer.

Available actions: `REQUEST_DOCS` , `MARK_FLAGGED` , `RECOMMEND_APPROVAL` .

Notifications: Customer notified via SMS/Email.

3.8 Underwriting (FR-UNDER)

Description: Underwriter performs risk assessment and final approval.

Actors: Underwriter.

Checks performed: Income-to-EMI ratio, CIBIL trend, employment validation, outstanding loans.

Outcome: `APPROVED` → generates sanction letter; `REJECTED` ; `REFER` (requires higher approval).

3.9 Sanction Letter Generation (FR-SANCT)

Description: Generate PDF sanction letter with loan terms, interest rate, processing fee, EMI schedule.

Actors: System, Customer.

Postconditions: Letter available for customer download and stored in document store.

3.10 Disbursement (FR-DISB)

Description: Disbursement officer completes final checks and initiates payout.

Actors: Disbursement Officer, Payment Gateway/Bank transfer system.

Checks: Sanction letter signed, KYC verified, account validation.

Postconditions: ledger updated, EMI schedule set, status updated to **DISBURSED**.

3.11 EMI Schedule & Repayment (FR-EMI)

Description: System calculates EMI schedule and records repayments.

Actors: Customer, Collection Team.

Payment methods: UPI, Netbanking, Debit Card, Auto-debit (mandate).

Edge cases: Partial payments, pre-payment, foreclosure — handled per business rules and fees.

3.12 Collections Management (FR-COL)

Description: Track overdue EMIs and manage reminders, escalations, and recovery status.

Actors: Collection Team.

Features: Aging buckets, reminder templates, manual follow-up logging.

3.13 Support & Ticketing (FR-SUP)

Description: Customers create tickets for queries; internal teams respond.

Actors: Customer, Support Agents.

SLA: Response within TBD hours (configurable).

3.14 Admin & Configuration (FR-ADMIN)

Description: Admin manages users, roles, loan products, interest rates, processing fees, thresholds.

Actors: Admin, Super Admin.

Audit: All changes logged.

3.15 Reporting & Dashboard (FR-REPORT)

Description: Role-based dashboards and reports (daily disbursal, overdue, KYC pending).

Actors: Admin, Managers.

4. External Interface Requirements

4.1 User interfaces (UI)

- **Customer portal:** Responsive SPA with multi-step forms, document uploader, dashboard.
- **Bank staff portal:** Role-based pages, list views, detail pages, action buttons, audit trail viewer.
- **Admin console:** Configuration screens, product management, user management.

4.2 Hardware interfaces

- Standard server hardware (or virtual machines): 4+ CPU, 8–16GB RAM for app server (scale horizontally). Database server with appropriate resources.

4.3 Software interfaces (APIs)

- **CIBIL API** — REST; input: PAN, name, DOB; output: score + bureau report.
- **PAN/Aadhaar verification API** — REST; input: PAN/Aadhaar; output: verification status.
- **SMS Gateway** — REST; input: message, mobile.
- **Email Service** — SMTP/REST.
- **Payment Gateway / Bank Transfer** — REST for UPI/netbanking/NEFT initiation and callbacks.

4.4 Communications protocols

- HTTPS/TLS 1.2+ for all external calls
 - REST/JSON for APIs
 - Secure MQ (optional) for internal asynchronous tasks (document processing, notifications)
-

5. Non-Functional Requirements (NFR)

5.1 Performance requirements

- API median response time < 500ms under normal load; 95th percentile < 2s.
- System supports 10,000 concurrent users (scale plan for more).
- Loan application submission throughput: 500 req/min.

5.2 Reliability & Availability

- Uptime: 99.9% (monthly).
- Database backups: daily full, hourly incremental.
- Disaster recovery RTO: 4 hours; RPO: 1 hour (configurable per bank policy).

5.3 Security

- OWASP Top 10 mitigations.
- Data encryption at rest (AES-256) for PII/documents.
- Encryption in transit (TLS).
- Role-based access controls + fine-grained permissions.
- Audit logging for all sensitive operations.
- Account lockout, password complexity, session timeout.
- Sensitive fields (Aadhaar/PAN) masked in UI and logs.

5.4 Scalability

- Horizontally scale app servers behind LB.
- Database vertical/horizontal scaling plan (read-replicas for reporting).
- Async workers for document processing and notifications.

5.5 Maintainability

- Modular package layout; layered architecture (controller → service → repository).
- Unit tests for 80%+ coverage on business logic; integration tests for critical flows.
- Clear coding standards and CI pipeline.

5.6 Portability

- Application packaged as executable JAR; can run on any platform with Java 17.

5.7 Compliance

- Must comply with RBI guidelines, KYC/AML rules, data retention policies, and local data privacy laws.
-

6. System Architecture & Design Notes

6.1 High-level architecture

- Monolithic Spring Boot app with layered modules.
- Database (RDBMS) for transactional data.
- Encrypted document store for files.
- Notification subsystem (email/SMS) via adapters.
- Integration adapters for external APIs.

- Background scheduler for retries, reminders, reports (Quartz or Spring Scheduler).

6.2 Packages (Java)

- `com.bank.plms.auth`
- `com.bank.plms.customer`
- `com.bank.plms.loan`
- `com.bank.plms.document`
- `com.bank.plms.verification`
- `com.bank.plms.underwriting`
- `com.bank.plms.disbursement`
- `com.bank.plms.payment`
- `com.bank.plms.collection`
- `com.bank.plms.admin`
- `com.bank.plms.notification`
- `com.bank.plms.reporting`
- `com.bank.plms.audit`

6.3 Design patterns

- DTOs for API payloads
 - Repository (Spring Data JPA), Service, Controller layers
 - Strategy pattern for interest calculation & risk scoring rules
 - Factory pattern for document processors
 - Circuit breaker for external API calls (Resilience4j)
-

7. Data Design (High-Level)

7.1 Key entities (tables)

- **users** (user_id, username, password_hash, role, status, created_at)
- **customers** (customer_id, user_id, name, dob, address, mobile, email, pan_masked, aadhaar_masked, created_at)
- **loan_applications** (application_id, customer_id, amount, tenure_months, purpose, status, created_at, updated_at)
- **loan_products** (product_id, name, min_amount, max_amount, min_tenure, max_tenure, interest_rate_base, processing_fee)
- **documents** (document_id, application_id, type, filename, storage_path, checksum, status, uploaded_by, uploaded_at)
- **cibil_reports** (report_id, application_id, score, report_json, fetched_at)
- **sanction_letters** (sanction_id, application_id, pdf_path, issued_on)
- **disbursements** (disbursement_id, application_id, amount, bank_account, status, disbursed_at)
- **emi_schedule** (schedule_id, loan_id, installment_no, due_date, principal, interest, total, status)
- **transactions** (transaction_id, schedule_id, amount, method, status, txn_date)
- **audit_logs** (log_id, entity, entity_id, action, performed_by, timestamp, details)
- **tickets** (ticket_id, customer_id, subject, description, status, created_at, assigned_to)

7.2 Data integrity

- Foreign keys linking applications → customers → users.
 - Use transactions for operations that update multiple tables (e.g., approve + generate EMI + disburse).
 - Use optimistic locking (version column) for critical updates.
-

8. API Specification (Overview — to be expanded to OpenAPI)

All endpoints are HTTPS. JWT in Authorization header: Authorization: Bearer <token>. Request/response payloads use JSON.

8.1 Authentication

- `POST /api/v1/auth/register` — body: {mobile, email, password} → returns userId, OTP sent
- `POST /api/v1/auth/verify-otp` — body: {userId, otp}
- `POST /api/v1/auth/login` — body: {username, password} → returns accessToken
- `POST /api/v1/auth/forgot-password` — body: {email|mobile}

8.2 Customer

- `GET /api/v1/customers/{id}` — get profile
- `PUT /api/v1/customers/{id}` — update profile

8.3 Loan Application

- `POST /api/v1/applications` — submit application
- `GET /api/v1/applications/{id}` — get application details (role-based view)
- `GET /api/v1/applications?status=SUBMITTED` — list with filters

8.4 Documents

- `POST /api/v1/applications/{id}/documents` — multipart upload
- `GET /api/v1/applications/{id}/documents` — list docs
- `PUT /api/v1/documents/{docId}/status` — verification result

8.5 CIBIL & KYC

- `POST /api/v1/integrations/cibil/fetch` — trigger fetch for app

- `POST /api/v1/integrations/pan/verify` — returns verify result

8.6 Loan Actions (LO / Underwriter)

- `POST /api/v1/applications/{id}/action` — body: {action: APPROVE|REJECT|REQUEST_DOCS, comments}
- `GET /api/v1/applications/{id}/audit` — audit trail

8.7 Disbursement

- `POST /api/v1/applications/{id}/disburse` — initiate disbursement

8.8 Payments

- `POST /api/v1/emi/{scheduleId}/pay` — initiate payment
- `POST /api/v1/emi/webhook` — payment gateway callback

8.9 Admin

- `GET /api/v1/admin/reports` — report endpoints
 - `PUT /api/v1/admin/products/{id}` — update product/rates
-

9. Error Handling & Logging

9.1 Error model

Standard API error schema:

```
{
  "timestamp": "...",
  "status": 400,
  "error": "Bad Request",
  "code": "VALIDATION_ERROR",
  "message": "PAN is invalid",
```

```
"details": [...]  
}
```

9.2 Logging

- Use structured logs (JSON) for micro-ops.
 - Log levels: ERROR, WARN, INFO, DEBUG (configurable per environment).
 - Mask PII in logs.
 - Centralized log aggregator (ELK/EFK stack).
-

10. Security Requirements

- TLS 1.2+ everywhere.
 - JWT with appropriate expiry and refresh tokens.
 - Role-based method-level security (Spring Security @PreAuthorize).
 - Input validation & sanitization.
 - Parameterized queries to avoid SQL injection.
 - Encryption of PII at rest (DB column-level or vault).
 - Key management per bank policy (HashiCorp Vault or HSM).
 - Regular vulnerability scanning and periodic penetration testing.
-

11. Performance & Scalability Plan

- Horizontal scaling of app servers behind load balancer.
- Read-replicas for reporting queries; master for writes.
- Caching layer (Redis) for reference data and frequent reads (loan products, rate tables).
- Async queues (RabbitMQ/Kafka) for heavy background tasks.

- Connection pooling & DB tuning.
-

12. Backup & Recovery

- Daily full DB backup + hourly incremental; encrypted backups.
 - Periodic restore drills (quarterly).
 - Document store backups replication.
 - Disaster recovery environment with RTO/RPO as specified (4h/1h default).
-

13. Test Plan (Summary)

13.1 Types of testing

- Unit tests (JUnit) — aim 80%+ for critical modules.
- Integration tests — service + DB.
- End-to-end tests — key flows using Selenium/Playwright (customer apply to disbursal).
- Performance tests — JMeter / Gatling (load, stress).
- Security tests — SAST (SonarQube) + DAST.
- UAT — functional acceptance by business.

13.2 Acceptance test cases (examples)

- Submit loan with valid KYC → should reach `UNDER_REVIEW`.
 - CIBIL < 700 → application moves to `AUTO_REJECT` or flagged.
 - Sanction letter generated after underwriter approval.
 - Disbursement updates ledger and EMI schedule.
 - EMI payment via gateway updates transaction and schedule status.
-

14. Deployment & DevOps

14.1 Build & CI/CD

- CI pipeline: build → unit tests → static analysis → artifact.
- CD pipeline: deploy to dev → staging (integration tests) → production (approval).
- Use GitHub Actions / Jenkins.

14.2 Environments

- Dev, QA, Staging, Production.
- Secrets managed via Vault; no secrets in repo.

14.3 Monitoring & Alerts

- Health endpoints `/actuator/health`.
 - Metrics via Prometheus + Grafana.
 - Alerting for high error rate, latency, failed jobs.
-

15. Maintenance & Support

- Support SLA for production incidents (P1, P2 definitions).
 - On-call rotation for ops.
 - Runbooks for common tasks (restart, DB failover, replay jobs).
-

16. Data Retention & Privacy

- Store KYC documents for minimum statutory period (bank policy) — e.g., 7 years.
- Provide data access controls & deletion workflows as per law.

- Customers' consent stored for data processing and sharing.
-

17. Acceptance Criteria & Deliverables

Must be met for system acceptance:

- All functional flows (apply → disburse) work end-to-end in staging.
- Role-based access control validated.
- EMI calculations validated against sample amortization tables.
- Security assessment passed (no critical vulnerabilities).
- Performance meets SLA (95th percentile < 2s).
- Backup & DR verified.

Deliverables:

- Codebase, DB schema, deployment scripts.
 - API docs (OpenAPI).
 - Test suites and reports.
 - Runbooks and user manuals.
-

18. Traceability Matrix (high-level)

BRS Section	SRS Feature(s)
Registration & Login	FR-AUTH
Document upload	FR-DOC
KYC & CIBIL	FR-KYC, FR-CIBIL
Underwriting	FR-UNDER
Disbursement	FR-DISB
EMI repayment	FR-EMI
Admin config	FR-ADMIN

(Complete traceability matrix to be delivered as a spreadsheet linking each BRS requirement to SRS feature & test cases.)

19. Appendices (Artifacts to produce)

- OpenAPI (Swagger) spec for all endpoints.
 - ER Diagram & Physical DB schema (DDL).
 - Sequence diagrams for key flows (application submit, underwriting, disbursement, EMI payment).
 - UI wireframes for customer and staff portals.
 - Security checklist and checklist for RBI compliance.
-